C PROGRAMMING

MOCK INTERVIEW QUESTION BANK

200 Curated Questions for Campus Placement Preparation

Third Year — ECE | EEE | BME

Section	Question Count
1. Technical Interview Questions	50
2. Programming Logic & Pseudocode	40
3. Placement Coding Questions	60
4. Output Prediction Questions	30
5. Debugging Questions	20
TOTAL	**200**

SECTION 1

Technical Interview Questions (50 Questions)

Q1	**Basics of C**	Language Fundamentals	**Basic**	**Theory**

What are the key features of the C programming language?

Q2	**Basics of C**	Compilation Process	**Basic**	**Theory**

Explain the steps involved in compiling and executing a C program.

Q3	**Basics of C**	Header Files	**Basic**	**Theory**

What is the purpose of the #include directive and header files in C?

Q4	**Basics of C**	Keywords vs Identifiers	**Basic**	**Theory**

Differentiate between a keyword and an identifier in C.

Q5	**Data Types**	Data Type Sizes	**Basic**	**Theory**

What is the difference between int, float, and double data types?

Q6	**Data Types**	Type Modifiers	**Basic**	**Theory**

Explain the use of signed, unsigned, short, and long type modifiers.

Q7	**Data Types**	Type Casting	**Medium**	**Theory**

Differentiate between implicit and explicit type conversion with examples.

Q8	**Variables**	Declaration vs Definition	**Basic**	**Theory**

What is the difference between declaration and definition of a variable?

Q9	**Variables**	Scope and Lifetime	**Medium**	**Theory**

Explain scope and lifetime of a variable in C with an example.

Q10	**Operators**	Arithmetic Operators	**Basic**	**Theory**

What is the difference between the modulus operator and the division operator?

Q11	**Operators**	Relational Operators	**Basic**	**Theory**

Explain how relational operators are evaluated in C and what value they return.

Q12	**Operators**	Logical Operators	**Basic**	**Theory**

Differentiate between logical AND (&&) and bitwise AND (&) operators.

Q13	**Operators**	Assignment Operators	**Basic**	**Theory**

What are compound assignment operators? Give three examples.

Q14	**Operators**	Increment/Decrement	**Medium**	**Theory**

Explain the difference between pre-increment and post-increment operators (++i vs i++).

Q15	**Operators**	Bitwise Operators	**Medium**	**Theory**

Explain the working of bitwise AND, OR, XOR, and NOT operators with examples.

Q16	**Operators**	Bitwise Operators	**Medium**	**Theory**

What is the use of the left shift (<<) and right shift (>>) operators?

Q17	**Operators**	Operator Precedence	**Medium**	**Theory**

What is operator precedence and associativity? Why are they important?

Q18	**Decision Making**	if-else vs switch	**Basic**	**Theory**

When should a switch statement be preferred over multiple if-else statements?

Q19	**Decision Making**	Nested Conditions	**Basic**	**Theory**

What is a dangling else problem in nested if-else statements?

Q20	**Loops**	Loop Types	**Basic**	**Theory**

Differentiate between while, do-while, and for loops in C.

Q21	**Loops**	break and continue	**Basic**	**Theory**

Explain the difference between the break and continue statements.

Q22	**Loops**	Infinite Loops	**Basic**	**Theory**

How is an infinite loop created intentionally in C? Give an example.

Q23	**Functions**	Function Basics	**Basic**	**Theory**

What is a function prototype and why is it necessary?

Q24	**Functions**	Call by Value vs Reference	**Medium**	**Theory**

Differentiate between Call by Value and Call by Reference with an example.

Q25	**Functions**	Return Types	**Basic**	**Theory**

Can a function return more than one value in C? How can this be achieved?

Q26	**Functions**	Inline Functions	**Medium**	**Theory**

What is the significance of the inline keyword for functions in C?

Q27	**Recursion**	Recursion Basics	**Basic**	**Theory**

What is recursion? What are the base case and recursive case?

Q28	**Recursion**	Recursion vs Iteration	**Medium**	**Theory**

Compare recursion and iteration in terms of memory usage and performance.

Q29	**Recursion**	Stack Overflow	**Medium**	**Theory**

What causes a stack overflow error in a recursive function?

Q30	**Arrays**	Array Basics	**Basic**	**Theory**

Why are array indices in C always started from zero?

Q31	**Arrays**	Array vs Pointer	**Medium**	**Theory**

Differentiate between an array and a pointer in C.

Q32	**Arrays**	2D Arrays	**Medium**	**Theory**

How is a 2D array stored in memory? Explain row-major order.

Q33	**Arrays**	Array as Function Argument	**Medium**	**Theory**

Why does an array decay into a pointer when passed to a function?

Q34	**Strings**	String Basics	**Basic**	**Theory**

How are strings represented internally in C? What is the role of the null character?

Q35	**Strings**	String Functions	**Basic**	**Theory**

Differentiate between strcpy() and strncpy() functions.

Q36	**Strings**	gets vs fgets	**Medium**	**Theory**

Why is gets() considered unsafe? How does fgets() overcome this issue?

Q37	**Storage Classes**	Storage Class Types	**Basic**	**Theory**

List and briefly explain the four storage classes available in C.

Q38	**Storage Classes**	static keyword	**Medium**	**Theory**

What is the significance of the static keyword for a local variable?

Q39	**Storage Classes**	extern keyword	**Medium**	**Theory**

What is the purpose of the extern storage class specifier?

Q40	**Pointers**	Pointer Basics	**Basic**	**Theory**

What is a NULL pointer, and how is it different from an uninitialized pointer?

Q41	**Pointers**	Void Pointer	**Medium**	**Theory**

Differentiate between a NULL pointer and a void pointer.

Q42	**Pointers**	Pointer Arithmetic	**Medium**	**Theory**

Explain pointer arithmetic with an example of incrementing a pointer to an int array.

Q43	**Pointers**	Double Pointers	**Medium**	**Theory**

What is a pointer to a pointer? Give a practical use case.

Q44	**Pointers**	Function Pointers	**Medium**	**Theory**

What is a function pointer? Where is it commonly used?

Q45	**Dynamic Memory Allocation**	malloc vs calloc	**Basic**	**Theory**

Differentiate between malloc() and calloc() functions.

Q46	**Dynamic Memory Allocation**	realloc and free	**Medium**	**Theory**

Explain the purpose of realloc() and free() functions with an example scenario.

Q47	**Dynamic Memory Allocation**	Memory Leak	**Medium**	**Theory**

What is a memory leak? How can it be avoided in a C program?

Q48	**Structures**	Structure vs Union	**Medium**	**Theory**

Differentiate between a structure and a union in terms of memory allocation.

Q49	**Structures**	Nested Structures	**Medium**	**Theory**

What is a nested structure? Give an example of accessing its members.

Q50	**Unions**	Union Memory	**Medium**	**Theory**

How much memory does a union occupy? Explain with an example.

Q51	**Enumeration**	enum Basics	**Basic**	**Theory**

What is an enum in C? What are the default values assigned to enum constants?

SECTION 2

Programming Logic & Pseudocode (40 Questions)

Q51	**Programming Logic**	Prime Number	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Prime Number.

Q52	**Programming Logic**	Palindrome Number	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Palindrome Number.

Q53	**Programming Logic**	Armstrong Number	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Armstrong Number.

Q54	**Programming Logic**	Strong Number	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Strong Number.

Q55	**Programming Logic**	Perfect Number	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Perfect Number.

Q56	**Programming Logic**	Neon Number	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Neon Number.

Q57	**Programming Logic**	Reverse Number	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Reverse Number.

Q58	**Programming Logic**	Reverse Digits	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Reverse Digits.

Q59	**Programming Logic**	Count Digits	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Count Digits.

Q60	**Programming Logic**	Sum of Digits	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Sum of Digits.

Q61	**Programming Logic**	Product of Digits	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Product of Digits.

Q62	**Programming Logic**	Even/Odd Check	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to perform Even/Odd Check.

Q63	**Programming Logic**	Leap Year	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Leap Year.

Q64	**Programming Logic**	Factorial	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Factorial.

Q65	**Programming Logic**	Fibonacci Series	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Fibonacci Series.

Q66	**Programming Logic**	Power Calculation	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Power Calculation.

Q67	**Programming Logic**	GCD	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print GCD.

Q68	**Programming Logic**	LCM	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print LCM.

Q69	**Programming Logic**	Prime Numbers Between Two Numbers	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Prime Numbers Between Two Numbers.

Q70	**Programming Logic**	Palindrome String	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Palindrome String.

Q71	**Programming Logic**	Reverse String	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Reverse String.

Q72	**Programming Logic**	Count Vowels	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Count Vowels.

Q73	**Programming Logic**	Count Words	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Count Words.

Q74	**Programming Logic**	Linear Search	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to perform Linear Search.

Q75	**Programming Logic**	Binary Search	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to perform Binary Search.

Q76	**Programming Logic**	Bubble Sort	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to perform Bubble Sort.

Q77	**Programming Logic**	Selection Sort	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to perform Selection Sort.

Q78	**Programming Logic**	Insertion Sort	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to perform Insertion Sort.

Q79	**Programming Logic**	Second Largest Element	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Second Largest Element.

Q80	**Programming Logic**	Duplicate Elements	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Duplicate Elements.

Q81	**Programming Logic**	Pattern Printing (Triangle)	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Pattern Printing (Triangle).

Q82	**Programming Logic**	Star Patterns	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Star Patterns.

Q83	**Programming Logic**	Number Patterns	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Number Patterns.

Q84	**Programming Logic**	Alphabet Patterns	**Medium**	**Pseudocode**

Write the algorithm/pseudocode to find/print Alphabet Patterns.

Q85	**Programming Logic**	Sum of Array Elements	**Basic**	**Pseudocode**

Write the algorithm/pseudocode to find/print Sum of Array Elements.

Q86	**Programming Logic**	While vs For Loop	**Medium**	**Theory**

What is the difference between a while loop and a for loop? Explain with a scenario when you would prefer one over the other.

Q87	**Programming Logic**	Array vs Pointer Usage	**Medium**	**Theory**

What is the difference between an array and a pointer, and when should each be used while designing a program?

Q88	**Programming Logic**	SDLC Awareness	**Medium**	**Theory**

What is SDLC (Software Development Life Cycle)? Briefly explain its phases and why understanding it is useful for a programmer.

Q89	**Programming Logic**	Call by Value vs Call by Reference	**Medium**	**Theory**

What is the difference between call by value and call by reference? Explain a situation where call by reference is necessary.

Q90	**Programming Logic**	Compile-Time vs Run-Time Errors	**Medium**	**Theory**

What is the difference between a compile-time error and a run-time error? Give one example of each.

SECTION 3

Dry Run & Output Prediction — Hard Level (60 Questions)

Q91	Loops	Nested Loop Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int i, j, k = 0;
    for (i = 1; i <= 3; i++) {
        for (j = 1; j <= i; j++) {
            k += i * j;
        }
    }
    printf("%d", k);
    return 0;
}
```

Q92	Loops	Loop with Multiple Conditions	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int i = 0, sum = 0;
    while (i < 10) {
        if (i % 2 == 0 && i % 3 == 0) {
            sum += i;
        }
        i++;
    }
    printf("%d", sum);
    return 0;
}
```

Q93	Loops	For Loop with Comma Operator	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int i, j;
    for (i = 0, j = 5; i < j; i++, j--) {
        printf("%d-%d ", i, j);
    }
    return 0;
}
```

Q94	Functions	Nested Function Calls	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int square(int x) { return x * x; }
int cube(int x) { return x * square(x); }
int main() {
    printf("%d", cube(square(2)));
    return 0;
}
```

Q95	Functions	Function with Static Local Variable	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int counter() {
    static int c = 10;
    c -= 2;
    return c;
}
int main() {
    printf("%d %d %d", counter(), counter(), counter());
    return 0;
}
```

Q96	Functions	Function Returning Pointer	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int *getPtr(int *arr) {
    return arr + 2;
}
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = getPtr(arr);
    printf("%d", *p + *(p + 1));
    return 0;
}
```

Q97	Recursion	Mutual Style Recursion	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int fun(int n) {
    if (n <= 1) return 1;
    return n * fun(n - 1) + fun(n - 2);
}
int main() {
    printf("%d", fun(4));
    return 0;
```

```
}
```

Q98	Recursion	Recursion with Static Variable	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
void fun(int n) {
    static int sum = 0;
    if (n == 0) {
        printf("%d", sum);
        return;
    }
    sum += n;
    fun(n - 1);
}
int main() {
    fun(4);
    return 0;
}
```

Q99	Recursion	Recursion Returning Multiple Paths	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int fun(int a, int b) {
    if (a == 0) return b;
    return fun(a - 1, a + b);
}
int main() {
    printf("%d", fun(3, 2));
    return 0;
}
```

Q100	Arrays	Array with Pointer Arithmetic	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[] = {2, 4, 6, 8, 10};
    int *p = arr;
    printf("%d", *(p + 1) + *(arr + 3) - *(p + 4));
    return 0;
}
```

Q101	Arrays	2D Array Traversal	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    int sum = 0, i, j;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            if (i == j || i + j == 2) sum += arr[i][j];
    printf("%d", sum);
    return 0;
}
```

Q102	Arrays	Array Modification via Function	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
void modify(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        arr[i] = arr[i] * 2 + 1;
}
int main() {
    int arr[4] = {1, 2, 3, 4}, i;
    modify(arr, 4);
    for (i = 0; i < 4; i++) printf("%d ", arr[i]);
    return 0;
}
```

Q103	Strings	String with Pointer Manipulation	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    char str[] = "PLACEMENT";
    char *p = str;
    while (*p) {
        if ((p - str) % 2 == 0) printf("%c", *p);
        p++;
    }
    return 0;
}
```

Q104	Strings	String Length via Recursion Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int myLen(char *s) {
    if (*s == '\0') return 0;
```

```
    return 1 + myLen(s + 1);
}
int main() {
    printf("%d", myLen("SONA"));
    return 0;
}
```

Q105	Strings	Nested String Comparison	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <string.h>
int main() {
    char a[] = "abc", b[] = "abd";
    printf("%d", strcmp(a, b));
    return 0;
}
```

Q106	Pointers	Pointer to Pointer Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int a = 5;
    int *p = &a;
    int **q = &p;
    **q = **q + 10;
    *p = *p + 5;
    printf("%d", a);
    return 0;
}
```

Q107	Pointers	Array of Pointers with Loop	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int a = 1, b = 2, c = 3;
    int *arr[3] = {&a, &b, &c};
    int i, sum = 0;
    for (i = 0; i < 3; i++) sum += *arr[i] * (i + 1);
    printf("%d", sum);
    return 0;
}
```

Q108	Pointers	Pointer Arithmetic with 2D Array	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[2][3] = {{1,2,3},{4,5,6}};
    int *p = &arr[0][0];
    printf("%d", *(p + 2) + *(p + 4));
    return 0;
}
```

Q109	Structures	Structure Passed to Function	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Point { int x, y; };
void move(struct Point p) {
    p.x += 10;
    p.y += 10;
}
int main() {
    struct Point pt = {1, 2};
    move(pt);
    printf("%d %d", pt.x, pt.y);
    return 0;
}
```

Q110	Structures	Structure Passed by Pointer	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Point { int x, y; };
void move(struct Point *p) {
    p->x += 10;
    p->y += 10;
}
int main() {
    struct Point pt = {1, 2};
    move(&pt);
    printf("%d %d", pt.x, pt.y);
    return 0;
}
```

Q111	Structures	Array of Structures Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Item { int price; };
int main() {
    struct Item items[3] = {{100}, {200}, {300}};
```

```
    int i, total = 0;
    for (i = 0; i < 3; i++)
        total += items[i].price * (i + 1);
    printf("%d", total);
    return 0;
}
```

Q112	Dynamic Memory Allocation	malloc with Loop Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int *)malloc(5 * sizeof(int));
    int i;
    for (i = 0; i < 5; i++) p[i] = i * i;
    printf("%d", p[2] + p[4]);
    free(p);
    return 0;
}
```

Q113	Dynamic Memory Allocation	calloc Default Values	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int *)calloc(4, sizeof(int));
    p[1] = 5;
    printf("%d %d %d", p[0], p[1], p[2]);
    free(p);
    return 0;
}
```

Q114	Bitwise Operators	Combined Bitwise Expression	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int a = 12, b = 10;
    printf("%d", (a & b) | (a ^ b));
    return 0;
}
```

Q115	Bitwise Operators	Shift Operators Combined	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int a = 5;
    a = (a << 2) | (a >> 1);
    printf("%d", a);
    return 0;
}
```

Q116	Bitwise Operators	Bit Toggle Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int a = 9;
    a = a ^ (1 << 1);
    printf("%d", a);
    return 0;
}
```

Q117	Loops	Do-While with Break/Continue	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int i = 0, sum = 0;
    do {
        i++;
        if (i % 2 == 0) continue;
        if (i > 7) break;
        sum += i;
    } while (i < 10);
    printf("%d", sum);
    return 0;
}
```

Q118	Functions	Function with Default Global Access	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int x = 10;
int fun(int x) {
    x = x + 5;
    return x;
}
int main() {
    int result = fun(x);
    printf("%d %d", x, result);
    return 0;
```

```
}
```

Q119	Recursion	Recursive Array Sum via Pointer	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int sumArr(int *arr, int n) {
    if (n == 0) return 0;
    return *arr + sumArr(arr + 1, n - 1);
}
int main() {
    int arr[] = {1, 2, 3, 4};
    printf("%d", sumArr(arr, 4));
    return 0;
}
```

Q120	Arrays	Array Swap via Nested Loop	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[5] = {5, 3, 8, 1, 9}, i, j, temp;
    for (i = 0; i < 4; i++)
        for (j = 0; j < 4 - i; j++)
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
    printf("%d %d %d", arr[0], arr[2], arr[4]);
    return 0;
}
```

Q121	Strings	Character Case Conversion Loop	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    char str[] = "cGate2026";
    int i;
    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] >= 'a' && str[i] <= 'z') str[i] = str[i] - 32;
    }
    printf("%s", str);
    return 0;
}
```

Q122	Pointers	Function Pointer Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }
int main() {
    int (*fp)(int, int);
    fp = add;
    int r1 = fp(3, 4);
    fp = mul;
    int r2 = fp(3, 4);
    printf("%d %d", r1, r2);
    return 0;
}
```

Q123	Structures	Nested Structure Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Address { int pin; };
struct Student { char name[10]; struct Address addr; };
int main() {
    struct Student s = {"Vijay", {636005}};
    s.addr.pin += 5;
    printf("%d", s.addr.pin);
    return 0;
}
```

Q124	Dynamic Memory Allocation	realloc Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int *)malloc(2 * sizeof(int));
    p[0] = 1; p[1] = 2;
    p = (int *)realloc(p, 4 * sizeof(int));
    p[2] = 3; p[3] = 4;
    printf("%d", p[0] + p[1] + p[2] + p[3]);
    free(p);
    return 0;
}
```

Q125	Loops	Multiple Nested Loop with Break	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
```

```
int main() {
    int i, j, count = 0;
    for (i = 1; i <= 3; i++) {
        for (j = 1; j <= 3; j++) {
            if (j == 2) break;
            count++;
        }
    }
    printf("%d", count);
    return 0;
}
```

Q126	Functions	Recursive Function with Loop Inside	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int fun(int n) {
    int i, s = 0;
    if (n == 0) return 0;
    for (i = 1; i <= n; i++) s += i;
    return s + fun(n - 1);
}
int main() {
    printf("%d", fun(2));
    return 0;
}
```

Q127	Bitwise Operators	Bit Counting Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int n = 13, count = 0;
    while (n) {
        count += n & 1;
        n >>= 1;
    }
    printf("%d", count);
    return 0;
}
```

Q128	Arrays	Array Search with Sentinel	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[] = {4, 7, 2, 9, 5}, key = 9, i, pos = -1;
    for (i = 0; i < 5; i++) {
```

```
        if (arr[i] == key) {
            pos = i;
            break;
        }
    }
    printf("%d", pos);
    return 0;
}
```

Q129	Strings	Recursive Palindrome Check Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <string.h>
int isPal(char *s, int l, int r) {
    if (l >= r) return 1;
    if (s[l] != s[r]) return 0;
    return isPal(s, l + 1, r - 1);
}
int main() {
    char str[] = "MADAM";
    printf("%d", isPal(str, 0, strlen(str) - 1));
    return 0;
}
```

Q130	Pointers	Pointer to Structure Array	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Emp { int id, sal; };
int main() {
    struct Emp e[3] = {{1,1000},{2,2000},{3,3000}};
    struct Emp *p = e;
    printf("%d", (p + 1)->sal + p->id);
    return 0;
}
```

Q131	Loops	Loop Modifying Counter Inside Body	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i < 10; i++) {
        if (i == 3) i += 2;
        printf("%d ", i);
```

```
    }
    return 0;
}
```

Q132	Functions	Function Modifying Array via Pointer Return	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
void fill(int *arr, int n) {
    int i;
    for (i = 0; i < n; i++) *(arr + i) = (i + 1) * (i + 1);
}
int main() {
    int arr[4];
    fill(arr, 4);
    printf("%d", arr[1] + arr[3]);
    return 0;
}
```

Q133	Recursion	Recursive Power with Two Branches	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int power(int b, int e) {
    if (e == 0) return 1;
    if (e % 2 == 0) return power(b * b, e / 2);
    return b * power(b, e - 1);
}
int main() {
    printf("%d", power(2, 5));
    return 0;
}
```

Q134	Arrays	Two Array Interaction	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int a[3] = {1, 2, 3}, b[3], i;
    for (i = 0; i < 3; i++) b[i] = a[i] * a[2 - i];
    printf("%d %d %d", b[0], b[1], b[2]);
    return 0;
}
```

Q135	Structures	Union inside Structure Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
union Val { int i; char c; };
struct Box { union Val v; int tag; };
int main() {
   struct Box b;
   b.v.i = 321;
   b.tag = b.v.c;
   printf("%d", b.tag);
   return 0;
}
```

Q136	Dynamic Memory Allocation	2D Dynamic Array Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   int **arr, i, j;
   arr = (int **)malloc(2 * sizeof(int *));
   for (i = 0; i < 2; i++) arr[i] = (int *)malloc(2 * sizeof(int));
   for (i = 0; i < 2; i++)
      for (j = 0; j < 2; j++)
         arr[i][j] = i + j;
   printf("%d", arr[0][1] + arr[1][0] + arr[1][1]);
   return 0;
}
```

Q137	Bitwise Operators	Swap Using XOR Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
   int a = 6, b = 11;
   a = a ^ b;
   b = a ^ b;
   a = a ^ b;
   printf("%d %d", a, b);
   return 0;
}
```

Q138	Loops	While Loop with Compound Update	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
   int i = 1, sum = 0;
   while (i <= 20) {
```

```
        sum += i;
        i *= 2;
    }
    printf("%d", sum);
    return 0;
}
```

Q139	Functions	Function Returning Struct Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Pair { int a, b; };
struct Pair make(int x, int y) {
    struct Pair p;
    p.a = x + y;
    p.b = x - y;
    return p;
}
int main() {
    struct Pair r = make(10, 4);
    printf("%d %d", r.a, r.b);
    return 0;
}
```

Q140	Recursion	Recursive Digit Reversal Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int rev(int n, int r) {
    if (n == 0) return r;
    return rev(n / 10, r * 10 + n % 10);
}
int main() {
    printf("%d", rev(4321, 0));
    return 0;
}
```

Q141	Arrays	Array Rotation Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5}, temp, i;
    temp = arr[0];
    for (i = 0; i < 4; i++) arr[i] = arr[i + 1];
    arr[4] = temp;
    printf("%d %d %d", arr[0], arr[2], arr[4]);
    return 0;
```

```
}
```

Q142	Strings	String Token Count Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    char str[] = "C is fun to learn";
    int i, words = 1;
    for (i = 0; str[i] != '\0'; i++) {
        if (str[i] == ' ') words++;
    }
    printf("%d", words);
    return 0;
}
```

Q143	Pointers	Pointer Arithmetic with Increment Chain	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[] = {10, 20, 30, 40};
    int *p = arr;
    printf("%d ", *p++);
    printf("%d ", *p);
    printf("%d", *++p);
    return 0;
}
```

Q144	Structures	Structure Array with Function Update	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
struct Stu { int marks; };
void grade(struct Stu s[], int n) {
    int i;
    for (i = 0; i < n; i++) s[i].marks += 5;
}
int main() {
    struct Stu s[3] = {{50},{60},{70}};
    grade(s, 3);
    printf("%d %d %d", s[0].marks, s[1].marks, s[2].marks);
    return 0;
}
```

Q145	Dynamic Memory Allocation	Freed Pointer Reuse Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int *)malloc(sizeof(int));
    *p = 25;
    int val = *p;
    free(p);
    printf("%d", val);
    return 0;
}
```

Q146	Bitwise Operators	Checking Power of Two	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int n = 16;
    printf("%d", (n & (n - 1)) == 0);
    return 0;
}
```

Q147	Loops	For Loop with Decreasing Step	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int i, product = 1;
    for (i = 5; i >= 1; i -= 2) {
        product *= i;
    }
    printf("%d", product);
    return 0;
}
```

Q148	Functions	Recursive Function Called in Loop	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int fact(int n) {
    return (n <= 1) ? 1 : n * fact(n - 1);
}
int main() {
    int i, sum = 0;
    for (i = 1; i <= 3; i++) sum += fact(i);
    printf("%d", sum);
```

```
    return 0;
}
```

Q149	Arrays	Frequency Count Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 2, 3, 3, 3}, i, j, count;
    for (i = 0; i < 6; i++) {
        count = 0;
        for (j = 0; j < 6; j++) if (arr[j] == arr[i]) count++;
        if (arr[i] == 3) { printf("%d", count); break; }
    }
    return 0;
}
```

Q150	Recursion	Recursive GCD Dry Run	Hard	Dry Run

Dry-run the following program step by step and write down the exact output:

```
#include <stdio.h>
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
int main() {
    printf("%d", gcd(36, 24));
    return 0;
}
```

SECTION 4

Output Prediction Questions (30 Questions)

Q151	Increment & Decrement	Pre vs Post	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int i = 5;
    printf("%d %d %d", i++, ++i, i);
    return 0;
}
```

Q152	Operators	Operator Precedence	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a = 5, b = 2, c;
    c = a / b * b + a % b;
    printf("%d", c);
    return 0;
}
```

Q153	Loops	For Loop	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i < 5; i++) {
        if (i == 3) break;
        printf("%d ", i);
    }
    return 0;
}
```

Q154	Loops	Continue Statement	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 5; i++) {
        if (i % 2 == 0) continue;
        printf("%d ", i);
    }
```

```
    return 0;
}
```

Q155	Nested Loops	Nested For Loop	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int i, j;
    for (i = 1; i <= 3; i++) {
        for (j = 1; j <= i; j++)
            printf("%d", j);
        printf("\n");
    }
    return 0;
}
```

Q156	Functions	Default Argument Behaviour	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int modify(int x) {
    x = x + 10;
    return x;
}
int main() {
    int a = 5;
    modify(a);
    printf("%d", a);
    return 0;
}
```

Q157	Functions	Return by Reference	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
void modify(int *x) {
    *x = *x + 10;
}
int main() {
    int a = 5;
    modify(&a);
    printf("%d", a);
    return 0;
}
```

Q158	Recursion	Recursive Sum	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int fun(int n) {
    if (n == 0) return 0;
    return n + fun(n - 1);
}
int main() {
    printf("%d", fun(4));
    return 0;
}
```

Q159	Recursion	Recursion Order	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
void printNum(int n) {
    if (n == 0) return;
    printf("%d ", n);
    printNum(n - 1);
}
int main() {
    printNum(3);
    return 0;
}
```

Q160	Pointers	Pointer Dereference	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a = 10;
    int *p = &a;
    *p = *p + 5;
    printf("%d", a);
    return 0;
}
```

Q161	Pointers	Pointer Arithmetic	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int arr[] = {10, 20, 30, 40};
    int *p = arr;
    printf("%d", *(p + 2));
    return 0;
```

```
}
```

Q162	Pointers	Double Pointer	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a = 20;
    int *p = &a;
    int **q = &p;
    printf("%d", **q);
    return 0;
}
```

Q163	Arrays	Array Indexing	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    printf("%d", arr[2] + arr[4]);
    return 0;
}
```

Q164	Arrays	2D Array Access	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int arr[2][2] = {{1, 2}, {3, 4}};
    printf("%d", arr[0][1] + arr[1][0]);
    return 0;
}
```

Q165	Strings	String Length	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Placement";
    printf("%d", (int)strlen(str));
    return 0;
}
```

Q166	Strings	String Concatenation	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
#include <string.h>
int main() {
    char a[20] = "Sona";
    char b[] = "College";
    strcat(a, b);
    printf("%s", a);
    return 0;
}
```

Q167	Storage Classes	Static Variable	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
void counter() {
    static int count = 0;
    count++;
    printf("%d ", count);
}
int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Q168	Storage Classes	Auto Variable Scope	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int x = 10;
    {
        int x = 20;
        printf("%d ", x);
    }
    printf("%d", x);
    return 0;
}
```

Q169	Bitwise Operators	Left Shift	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
```

```
    int a = 3;
    printf("%d", a << 2);
    return 0;
}
```

Q170	Bitwise Operators	XOR Operator	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a = 5, b = 3;
    printf("%d", a ^ b);
    return 0;
}
```

Q171	Switch	Fall-through Behaviour	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int x = 2;
    switch (x) {
        case 1: printf("One");
        case 2: printf("Two");
        case 3: printf("Three"); break;
        default: printf("Default");
    }
    return 0;
}
```

Q172	Variable Scope	Global vs Local	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int x = 100;
void display() {
    int x = 10;
    printf("%d ", x);
}
int main() {
    display();
    printf("%d", x);
    return 0;
}
```

Q173	Operators	Logical Operators	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a = 0, b = 5;
    printf("%d", a && b);
    printf(" %d", a || b);
    return 0;
}
```

Q174	Operators	Comma Operator	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a;
    a = (3, 4, 5);
    printf("%d", a);
    return 0;
}
```

Q175	Loops	While Loop with Break	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int i = 0;
    while (1) {
        printf("%d ", i);
        i++;
        if (i == 3) break;
    }
    return 0;
}
```

Q176	Functions	Recursion with Multiple Returns	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int fun(int a, int b) {
    if (b == 0) return 0;
    return a + fun(a, b - 1);
}
int main() {
    printf("%d", fun(3, 4));
    return 0;
}
```

Q177	Pointers	Array of Pointers	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int *p[2];
    p[0] = &a;
    p[1] = &b;
    printf("%d", *p[0] + *p[1]);
    return 0;
}
```

Q178	Structures	Structure Member Access	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
struct Student {
    int marks;
};
int main() {
    struct Student s = {85};
    s.marks += 10;
    printf("%d", s.marks);
    return 0;
}
```

Q179	Unions	Union Memory Sharing	Medium	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
union Data {
    int i;
    char c;
};
int main() {
    union Data d;
    d.i = 65;
    printf("%c", d.c);
    return 0;
}
```

Q180	Enumeration	enum Default Values	Basic	Output Prediction

Predict the output of the following program:

```
#include <stdio.h>
enum Day { MON, TUE, WED };
int main() {
```

```
    enum Day d = WED;
    printf("%d", d);
    return 0;
}
```

SECTION 5

Debugging Questions (20 Questions)

Q181	Semicolon Errors	Misplaced Semicolon	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i < 5; i++);
    {
        printf("%d ", i);
    }
    return 0;
}
```

Q182	Semicolon Errors	Missing Semicolon	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int a = 5
    printf("%d", a);
    return 0;
}
```

Q183	Braces	Missing Braces in If-Else	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int a = 10;
    if (a > 5)
        printf("Greater");
        printf("Than 5");
    return 0;
}
```

Q184	Braces	Unbalanced Braces	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i < 3; i++) {
        printf("%d ", i);
    return 0;
```

```
}
```

Q185	Pointers	Uninitialized Pointer	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int *p;
    *p = 10;
    printf("%d", *p);
    return 0;
}
```

Q186	Pointers	Wrong Dereference	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int a = 10;
    int *p;
    p = a;
    printf("%d", *p);
    return 0;
}
```

Q187	Arrays	Array Index Out of Bounds	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int i;
    for (i = 0; i <= 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Q188	Arrays	Wrong Array Initialization	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int arr[5];
    arr = {1, 2, 3, 4, 5};
    printf("%d", arr[0]);
```

```
    return 0;
}
```

Q189	Strings	Buffer Overflow	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[5];
    strcpy(str, "Placement");
    printf("%s", str);
    return 0;
}
```

Q190	Strings	Missing String Header	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    char str1[20] = "Hello";
    char str2[] = "World";
    strcat(str1, str2);
    printf("%s", str1);
    return 0;
}
```

Q191	Functions	Missing Return Statement	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int square(int n) {
    int result = n * n;
}
int main() {
    printf("%d", square(5));
    return 0;
}
```

Q192	Functions	Function Prototype Mismatch	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int add(int a, int b);
int main() {
    printf("%d", add(5, 10, 15));
```

```
    return 0;
}
int add(int a, int b) {
    return a + b;
}
```

Q193	Recursion	Missing Base Case	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int factorial(int n) {
    return n * factorial(n - 1);
}
int main() {
    printf("%d", factorial(5));
    return 0;
}
```

Q194	Recursion	Incorrect Recursive Call	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int sum(int n) {
    if (n == 0)
        return 0;
    return n + sum(n);
}
int main() {
    printf("%d", sum(5));
    return 0;
}
```

Q195	malloc()	Missing Header for malloc	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int *p = malloc(5 * sizeof(int));
    p[0] = 10;
    printf("%d", p[0]);
    return 0;
}
```

Q196	malloc()	Not Freeing Memory / NULL Check	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *p = (int *)malloc(5 * sizeof(int));
    p[10] = 100;
    printf("%d", p[10]);
    return 0;
}
```

Q197	scanf()	Missing Address Operator	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int a;
    printf("Enter a number: ");
    scanf("%d", a);
    printf("%d", a);
    return 0;
}
```

Q198	printf()	Format Specifier Mismatch	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    float a = 10.5;
    printf("%d", a);
    return 0;
}
```

Q199	Bitwise Operators	Confusing && with &	Medium	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
int main() {
    int a = 6, b = 3;
    if (a & b)
        printf("True");
    else
        printf("False");
    return 0;
}
```

Q200	Structures	Wrong Member Access Operator	Basic	Debugging

Identify the error(s), correct the code, and explain why the error occurs:

```
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
};
int main() {
    struct Student s;
    s->roll = 1;
    printf("%d", s.roll);
    return 0;
}
```